Python Full Stack Developer Course Syllabus

Abstract

This syllabus is designed to equip learners with the skills to become proficient Python Full Stack Developers, covering front-end, back-end, databases, version control, deployment, and project work. The course duration is typically 6–8 months, with hands-on projects to build a portfolio.

1 Module 1: Introduction to Python Programming

Duration: 2–3 weeks
Objective: Build a strong foundation in Python programming fundamentals.

1.1 Python Basics

- Syntax, variables, and data types (integers, floats, strings, booleans).
- Input/output operations.
- Typecasting and basic operators.
- **Example**: Calculate the area of a rectangle.

```
length = float(input("Enter length: "))
width = float(input("Enter width: "))
area = length * width
print(f"The area of the rectangle is {area} square units")
```

1.2 Control Flow

- Conditional statements (if, elif, else).
- Loops (for, while).
- **Example**: Check if a number is even or odd.

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print(f"{num} is even")
else:
    print(f"{num} is odd")
```

1.3 Data Structures

- Lists, tuples, sets, and dictionaries.
- List comprehensions and generators.
- **Example**: Create a list of squares for numbers 1–10.

```
squares = [x**2 for x in range(1, 11)]
print(squares)  % Output: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

1.4 Functions and Modules

- Defining functions, parameters, and return statements.
- Importing and creating modules.
- **Example**: Check if a string is a palindrome.

```
def is_palindrome(text):
    return text.lower() == text.lower()[::-1]
print(is_palindrome("Radar"))  % Output: True
```

1.5 Exception Handling

- Try-except-else-finally blocks.
- Custom exceptions.
- **Example**: Handle division by zero.

```
try:
    a = int(input("Enter numerator: "))
    b = int(input("Enter denominator: "))
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero!")
else:
    print(f"Result: {result}")
```

Project: Build a simple calculator application that performs basic arithmetic operations based on user input.

2 Module 2: Front-End Development

Duration: 3–4 weeks
Objective: Learn to create responsive and interactive user interfaces.

2.1 HTML Fundamentals

- Tags, attributes, and semantic HTML.
- Forms, tables, and multimedia elements.

- **Example**: Create a simple registration form.

```
<form>
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required>
    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required
        >
    <button type="submit">Register</button>
</form>
```

2.2 CSS3 Styling

- Selectors, properties, and box model.
- Flexbox, Grid, and responsive design.
- **Example**: Style a navigation bar.

```
nav {
    display: flex;
    justify-content: space-around;
    background-color: #333;
    padding: 10px;
}
nav a {
    color: white;
    text-decoration: none;
}
```

2.3 JavaScript Basics

- Variables, functions, arrays, and objects.
- DOM manipulation and event handling.
- **Example**: Toggle visibility of an element.

```
document.getElementById("toggleBtn").addEventListener("click",
    function() {
    let element = document.getElementById("content");
    element.style.display = element.style.display === "none" ? "
        block" : "none";
});
```

2.4 Front-End Frameworks (React)

- Components, props, and state.
- React hooks (useState, useEffect).
- **Example**: Create a counter component.

```jsx
import React, { useState } from 'react';
function Counter() {
    const [count, setCount] = useState(0);
    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => setCount(count + 1)}>Increment
               </button>
        </div>
    );
}
export default Counter;
```

Project: Build a responsive portfolio website using HTML, CSS, Bootstrap, and React, featuring a homepage, about section, and contact form.

3 Module 3: Back-End Development with Python

Duration: 3–4 weeks
Objective: Master server-side programming using Python frameworks.

3.1 Introduction to Web Development

- Client-server architecture.
- HTTP methods (GET, POST).
- **Example**: Simple Flask app to handle GET requests.

```python
from flask import Flask
app = Flask(__name__)
@app.route('/')
def home():
    return "Welcome to my Flask app!"
if __name__ == '__main__':
    app.run(debug=True)
```

3.2 Flask Framework

- Routing, templates (Jinja2), and form handling.
- RESTful APIs.
- **Example**: Create an API endpoint to fetch user data.

```python
from flask import Flask, jsonify
app = Flask(__name__)
users = [{"id": 1, "name": "Alice"}, {"id": 2, "name": "Bob"}]
@app.route('/api/users', methods=['GET'])
def get_users():
    return jsonify(users)
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

3.3 Django Framework

- Models, views, and templates.
- Django ORM and admin panel.
- **Example**: Create a blog model.

```
from django.db import models
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return self.title
```

3.4 API Development

- RESTful principles and Django REST Framework.
- JWT authentication.
- **Example**: Create a POST endpoint for user registration.

```
from rest_framework.views import APIView
from rest_framework.response import Response
class RegisterView(APIView):
    def post(self, request):
        username = request.data.get('username')
        return Response({"message": f"User {username} registered"
            })
```

Project: Develop a to-do list application using Flask or Django, with features for creating, updating, and deleting tasks.

4 Module 4: Database Management

Duration: 2–3 weeks
Objective: Learn to manage and integrate databases with web applications.

4.1 SQL Databases (MySQL/SQLite)

- CRUD operations (Create, Read, Update, Delete).
- Joins and indexing.
- **Example**: Query to fetch all users from a table.

```
SELECT * FROM users WHERE age > 18;
```

4.2 NoSQL Databases (MongoDB)

- Document-based storage and queries.
- MongoDB with Python (PyMongo).
- **Example**: Insert a document into MongoDB.

```
from pymongo import MongoClient
client = MongoClient('localhost', 27017)
db = client['mydb']
collection = db['users']
collection.insert_one({"name": "Alice", "age": 25})
```

4.3 ORM (Object-Relational Mapping)

- Django ORM for SQL databases.
- **Example**: Fetch all posts using Django ORM.

```
from myapp.models import Post
posts = Post.objects.all()
```

Project: Build a database-driven e-commerce application with product listings and user authentication.

5 Module 5: Version Control and Collaboration

Duration: 1–2 weeks
Objective: Master version control for collaborative development.

5.1 Git Basics

- Repositories, commits, branches, and merges.
- **Example**: Create a new branch and commit changes.

```
git branch feature-branch
git checkout feature-branch
git add .
git commit -m "Added new feature"
```

5.2 GitHub Workflow

- Push, pull, and pull requests.
- Resolving merge conflicts.
- **Example**: Push code to GitHub.

```
git push origin feature-branch
```

Project: Collaborate on a group project using GitHub, contributing code and resolving conflicts.

6 Module 6: Deployment and DevOps

Duration: 2–3 weeks
Objective: Learn to deploy and scale web applications.

6.1 Deployment Basics

- Deploying to Heroku or AWS.
- **Example**: Deploy a Flask app to Heroku.

```
heroku create myapp
git push heroku main
```

6.2 Containerization (Docker)

- Creating and running Docker containers.
- **Example**: Dockerfile for a Flask app.

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

6.3 CI/CD Pipelines

- Setting up automated testing and deployment.
- **Example**: GitHub Actions workflow.

```
name: CI Pipeline
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run tests
        run: pytest
```

Project: Deploy a full-stack application to a cloud platform like AWS or Heroku.

7 Module 7: Capstone Project

Duration: 3–4 weeks
Objective: Apply all learned concepts to build a real-world application.

- Design and develop a complete full-stack application.
- Integrate front-end, back-end, and database components.

- Deploy the application and create a portfolio.

Example Project: Build an e-commerce website with:

- **Front-end**: React-based UI for product browsing and cart.
- **Back-end**: Django/Flask API for user authentication and order processing.
- **Database**: MySQL for product data and MongoDB for user reviews.
- **Deployment**: Host on AWS with Docker.
- **Version Control**: Manage code on GitHub.

Deliverables:

- Functional web application.
- Documentation and portfolio entry.
- Presentation of the project to peers or instructors.

8 Additional Topics

8.1 Web Scraping

- Using libraries like BeautifulSoup and Scrapy.
- **Example**: Scrape a website for product prices.

```
from bs4 import BeautifulSoup
import requests
response = requests.get("https://example.com")
soup = BeautifulSoup(response.text, 'html.parser')
prices = soup.find_all('span', class_='price')
for price in prices:
    print(price.text)
```

8.2 Testing and Security

- Unit testing with pytest.
- Securing APIs with JWT and input validation.
- **Example**: Write a unit test for a function.

```
import pytest
def add(a, b):
    return a + b
def test_add():
    assert add(2, 3) == 5
```

8.3 Soft Skills

- Collaboration, communication, and technical interview preparation.

9 Learning Path Recommendations

- **Practice**: Complete at least 2–3 mini-projects per module.
- **Portfolio**: Host projects on GitHub to showcase to employers.
- **Continuous Learning**: Stay updated with trends via communities and webinars.
- **Certifications**: Earn certifications from platforms like Udemy or TalentSprint.

Estimated Duration: 6–8 months (20–25 hours/week).
Career Opportunities: Full Stack Developer, Web Developer, Software Engineer, UI/UX Designer, and more.